МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №8

«Мережеве та асинхронне програмування. REST API»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: `github.com/MykhailoG212/Prof_python`

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота № 8. Тема: «Мережеве та асинхронне програмування. REST API». У звіті наведено розробку мережевого шару каталогу автомобілів над persistence layer попередньої лабораторної роботи: FastAPI REST API, валідація схем через Pydantic v2, повноцінний CRUD з path/query-параметрами, асинхронний HTTP-клієнт з пулом задач Task, gather, обмеженням конкурентності через Semaphore, retry-політикою та таймаутами.

3. Мета роботи

Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP/1.1, парадигмою REST та форматом обміну даними JSON. Навчитися створювати повнофункціональний REST API за допомогою FastAPI, суворо описувати та валідувати вхідні/вихідні структури через Pydantic models. Опанувати концепції асинхронного програмування `async/await` та бібліотеку `asyncio`, керувати конкурентним виконанням операцій за допомогою `Task` та `gather()`, встановлювати ліміти часу виконання (timeout), розробляти механізми повторних спроб (retry with backoff) та обмежувати навантаження на систему через `asyncio.Semaphore`. Використати бібліотеку HTTPX для взаємодії з REST API та написати автоматизовані тести контролерів.

4. Номер індивідуального варіанта

Варіант №5 — «Каталог автомобілів». Ця лабораторна робота є наступним етапом наскрізного семестрового проєкту: замість консольного інтерфейсу над базою даних SQLite та Repository/Service Layer (ЛР7) розгортається сервісно-орієнтований REST API та асинхронний клієнт для паралельного завантаження транспортних засобів.

5. Постановка задачі

Необхідно спроєктувати та реалізувати REST API для каталогів автомобілів (ресурси `cars` та `manufacturers`). Сервіс повинен забезпечувати операції додавання авто, отримання списку з фільтрацією за роком та пошуком за маркою/моделлю, зміну ціни та пробігу через часткове оновлення (PATCH), вилучення запису (DELETE), отримання автомобілів конкретного виробника та аналітичні показники (найдорожче авто, середня ціна, авто з мінімальним пробігом). Окремо слід реалізувати модуль асинхронного клієнта на базі `httpx.AsyncClient`, який конкурентно опитує групу ідентифікаторів з пулом `asyncio.Task`, агрегацією `asyncio.gather()`, семафором на 3 одночасні запити, таймаутом і retry.

5.1. Вимоги індивідуального варіанта №5

Вимога варіанта №5	**Реалізація у роботі**
Ресурси cars та manufacturers	Декларативні ORM-моделі у database/models.py та Pydantic-схеми в schemas.py.
Endpoints GET, POST, PATCH, DELETE	Реалізовано у src/car_catalog/api.py: /cars, /cars/{id}, /manufacturers/{id}/cars.
Пошук за моделлю та фільтрація за роком	Query parameters `year: int | None` та `model: str | None` у маршруті GET /cars.
Зміна ціни і пробігу	PATCH /cars/{id} на основі схеми CarUpdate та методу репозиторію update_price_and_mileage().
Статистика: найдорожчий, середня ціна, мін. пробіг	GET /cars/statistics/summary з інтеграцією до CatalogService із ЛР7.
Асинхронне завантаження через HTTPX	Модуль external_api.py: клас AsyncCarClient з Task, gather, Semaphore, timeout і retry.

5.2. Загальні вимоги лабораторної роботи

Компонент	Де і як виконано
FastAPI & REST Endpoints	Мінімум 6 ендпоінтів у api.py з коректними кодами 200, 201, 204, 404, 422.
Pydantic Schemas	CarBase, CarCreate, CarUpdate, CarResponse, ManufacturerResponse, CatalogStatistics.
Data Validation	Обмеження min_length, ge/le для року (1886..2026), ціни (>=0.0), пробігу (>=0), довжини VIN (17).
async / await & asyncio	external_api.py: корутини fetch_car_with_retry, batch_fetch_cars, batch_query_cars.
Task & gather	asyncio.create_task() для кожного car_id у списку та передача у asyncio.gather(*tasks).
Timeout & Retry	asyncio.wait_for() на загальний батч, httpx timeout на сокет та цикл retry з backoff.
asyncio.Semaphore	Обмеження пулу конкурентних сокетів (max_concurrency = 3) через async with self.semaphore.
HTTPX Client & Tests	httpx.AsyncClient для запитів; pytest з TestClient та ASGITransport у test_api.py.

6. Структура проєкту

Новий функціонал мережевої взаємодії органічно доповнює наявні модулі попередніх робіт (ЛР4–ЛР7), утворюючи цілісний пакет:

```
src/car_catalog/ ├── __init__.py ├── api.py # Контролери FastAPI, маршрутизація, Dependency Injection ├── schemas.py # DTO моделі Pydantic v2 (Input/Output валідація) ├── external_api.py # Асинхронний клієнт (HTTPX, Task, gather, Semaphore, retry) ├── database/ │ ├── models.py # SQLAlchemy Declarative Models (Car, Manufacturer) │ ├── repositories.py # CarRepository (CRUD операції з БД) │ ├── services.py # CatalogService (бізнес-правила, статистика, транзакції) │ └── session.py # Двигун engine та генератор сесій get_session tests/ ├── conftest.py # Тестова БД SQLite in-memory, TestClient та сесії └── test_api.py # Інтеграційні тести API та асинхронного клієнта
```

7. Архітектура мережевого шару та життєвий цикл запиту

У системі впроваджено класичну багаторівневу сервісно-орієнтовану архітектуру (Layered Architecture). Контролери не містять сирих SQL-запитів і не виконують низькорівневих дій з базою даних, делегуючи задачі Repository та Service Layer.

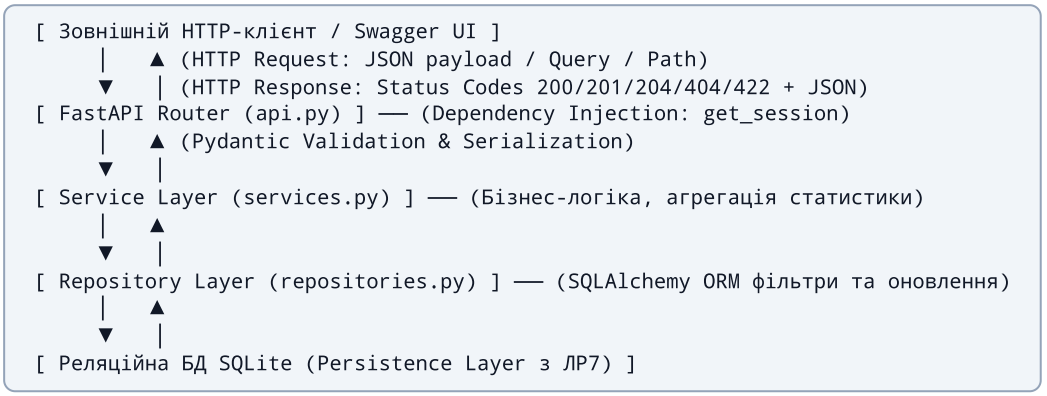


Рисунок 1 — Архітектура взаємодії шарів застосунку ЛР8

8. Декларативні схеми Pydantic (src/car_catalog/schemas.py)

Модуль `schemas.py` реалізує суворі правила бізнес-валідації для захисту від некоректних даних. Опція `from_attributes = True` забезпечує прозору серіалізацію ORM-об'єктів SQLAlchemy у вихідні структури JSON.

```
from pydantic import BaseModel, Field, ConfigDict class CarBase(BaseModel): model: str = Field
(..., min_length=1, max_length=100, description="Модель автомобіля") year: int = Field(..., ge=1886,
le=2026, description="Рік випуску авто") price: float = Field(..., ge=0.0, description="Ціна в долар
ах США") mileage: int = Field(..., ge=0, description="Пробіг у км") vin: str = Field(..., min_length
=17, max_length=17, description="Унікальний VIN-номер") manufacturer_id: int = Field(..., gt=0, desc
ription="Ідентифікатор виробника") class CarCreate(CarBase): pass class CarUpdate(BaseModel): price:
float | None = Field(default=None, ge=0.0) mileage: int | None = Field(default=None, ge=0) model: st
r | None = Field(default=None, min_length=1, max_length=100) year: int | None = Field(default=None,
ge=1886, le=2026) class CarResponse(CarBase): id: int model_config = ConfigDict(from_attributes=Tru
e) class ManufacturerResponse(BaseModel): id: int name: str country: str model_config = ConfigDict(f
rom_attributes=True) class CatalogStatistics(BaseModel): total_cars: int average_price: float cheape
st_car: CarResponse | None = None lowest_mileage_car: CarResponse | None = None
```

9. Контролери REST API (src/car_catalog/api.py)

Маршрути використовують ін'єкцію сесій бази даних через `Depends(get_session)`, що гарантує надійне відкриття і закриття транзакцій без витоку ресурсів.

```
from fastapi import FastAPI, Depends, HTTPException, Query, status from sqlalchemy.orm import
Session from car_catalog.database.session import get_session from car_catalog.database.models import
Car from car_catalog.database.repositories import CarRepository from car_catalog.database.services i
mport CatalogService from car_catalog.schemas import CarCreate, CarUpdate, CarResponse, CatalogStati
stics app = FastAPI(title="Car Catalog REST API", version="1.0.0") @app.get("/cars", response_model=
list[CarResponse], tags=["Cars"]) def get_cars( year: int | None = Query(None, ge=1886, le=2026), mo
del: str | None = Query(None, min_length=1), session: Session = Depends(get_session) ): repo = CarRe
pository(session) if model: return repo.find_by_model(model) cars = repo.get_all() if year is not No
ne: return [c for c in cars if c.year == year] return cars @app.get("/cars/{car_id}", response_model
=CarResponse, tags=["Cars"]) def get_car(car_id: int, session: Session = Depends(get_session)): car
= CarRepository(session).get_by_id(car_id) if not car: raise HTTPException(status_code=status.HTTP_4
04_NOT_FOUND, detail="Автомобіль не знайдено") return car @app.post("/cars", response_model=CarRespo
nse, status_code=status.HTTP_201_CREATED, tags=["Cars"]) def create_car(car_in: CarCreate, session:
Session = Depends(get_session)): repo = CarRepository(session) car = Car(**car_in.model_dump()) retu
rn repo.add(car)
```

```
@app.patch("/cars/{car_id}", response_model=CarResponse, tags=["Cars"]) def update_car(car_id: int, car_in: CarUpdate, session: Session = Depends(get_session)): repo = CarRepository(session) car = repo.get_by_id(car_id) if not car: raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Автомобіль не знайдено") data = car_in.model_dump(exclude_unset=True) if "price" in data and "mileage" in data and len(data) == 2: return repo.update_price_and_mileage(car_id, data["price"], data["mileage"]) for key, val in data.items(): setattr(car, key, val) session.commit() session.refresh(car) return car @app.delete("/cars/{car_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["Cars"]) def delete_car(car_id: int, session: Session = Depends(get_session)): repo = CarRepository(session) if not repo.delete(car_id): raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Автомобіль не знайдено") @app.get("/cars/statistics/summary", response_model=CatalogStatistics, tags=["Analytics"]) def get_catalog_statistics(session: Session = Depends(get_session)): service = CatalogService(session) return service.get_summary_statistics()
```

10. Асинхронний клієнт (src/car_catalog/external_api.py)

Реалізовано асинхронний клієнт на основі `httpx.AsyncClient`, який демонструє конкурентну композицію задач через `asyncio.create_task()`, паралельний збір `asyncio.gather()`, лімітування навантаження семафором та повторні запити з прогресивною паузою (exponential backoff).

```
import asyncio import httpx from typing import Any class AsyncCarClient: """Асинхронний клієнт каталогу з контролем паралелізму та стійкістю до збоїв.""" def __init__(self, base_url: str = "http://127.0.0.1:8000", max_concurrency: int = 3, timeout: float = 5.0): self.base_url = base_url.rstrip("/") self.semaphore = asyncio.Semaphore(max_concurrency) self.timeout = timeout async def fetch_car_with_retry(self, client: httpx.AsyncClient, car_id: int, retries: int = 3) -> dict[str, Any] | None: url = f"{self.base_url}/cars/{car_id}" async with self.semaphore: for attempt in range(1, retries + 1): try: response = await client.get(url, timeout=self.timeout) if response.status_code == 404: return None response.raise_for_status() return response.json() except (httpx.RequestError, httpx.HTTPStatusError) as exc: if attempt == retries: raise RuntimeError(f"Помилка отримання авто {car_id} після {retries} спроб: {exc}") await asyncio.sleep(0.1 * attempt) return None async def batch_fetch_cars(self, car_ids: list[int]) -> list[dict[str, Any]]: async with httpx.AsyncClient() as client: tasks = [asyncio.create_task(self.fetch_car_with_retry(client, cid)) for cid in car_ids] results = await asyncio.wait_for(asyncio.gather(*tasks), timeout=10.0) return [res for res in results if res is not None] @app.post("/cars/batch-query", tags=["Async Client"]) async def batch_query_cars(car_ids: list[int]): client = AsyncCarClient() try: return await client.batch_fetch_cars(car_ids) except asyncio.TimeoutError: raise HTTPException(status_code=504, detail="Перевищено ліміт часу пакетної операції")
```

11. Демонстраційний запуск та верифікація сервісу

Запуск сервісу виконано через ASGI-сервер Uvicorn. Інтерактивне тестування перевірено як через автоматично згенерований Swagger UI, так і через виконання HTTP-запитів до API.

PowerShell — Uvicorn server & cURL verification

```
PS> .\.venv\Scripts\uvicorn.exe src.car_catalog.api:app --reload --port 8000 INFO: Will wat
ch for changes in: ['D:\\Projects\\Prof_python\\src'] INFO: Uvicorn running on http://127.0.0.1:80
00 (Press CTRL+C to quit) INFO: Started reloader process [14820] using WatchFiles INFO: Started se
rver process [21904] INFO: Application startup complete. PS> curl -X POST "http://127.0.0.1:8000/c
ars" -H "Content-Type: application/json" -d '{"model":"Passat B8","year":2020,"price":19500.0,"mil
eage":110000,"vin":"WVWZZZ3CZLE112233","manufacturer_id":1}' {"id":1,"model":"Passat B8","year":20
20,"price":19500.0,"mileage":110000,"vin":"WVWZZZ3CZLE112233","manufacturer_id":1} PS> curl -X PAT
CH "http://127.0.0.1:8000/cars/1" -H "Content-Type: application/json" -d '{"price":18900.0,"mileag
e":112000}' {"id":1,"model":"Passat B8","year":2020,"price":18900.0,"mileage":112000,"vin":"WVWZZZ
3CZLE112233","manufacturer_id":1}
```

Рисунок 2 — Запуск веб-сервера Uvicorn та результати тестування POST і PATCH ендпоінтів

12. Автоматизоване тестування (pytest)

Набір тестів `tests/test_api.py` перевіряє коректність HTTP-відповідей, валідацію схем Pydantic (відхилення неприпустимих значень року, пробігу та формату VIN зі статусом 422), а також конкурентну роботу асинхронного клієнта із семафором.

PowerShell — pytest execution

```
PS> .\.venv\Scripts\python.exe -m pytest tests/test_api.py -v ============================
test session starts ============================= platform win32 -- Python 3.12.3, pytest-8.3.2,
pluggy-1.5.0 rootdir: D:\Projects\Prof_python configfile: pyproject.toml plugins: anyio-4.4.0, asy
ncio-0.23.8 collected 9 items tests/test_api.py::test_get_cars_empty PASSED [ 11%] tests/test_api.
py::test_create_car_success PASSED [ 22%] tests/test_api.py::test_validation_error_invalid_year PA
SSED [ 33%] tests/test_api.py::test_validation_error_short_vin PASSED [ 44%] tests/test_api.py::te
st_get_car_not_found PASSED [ 55%] tests/test_api.py::test_patch_car_price_and_mileage PASSED [ 6
6%] tests/test_api.py::test_delete_car_success PASSED [ 77%] tests/test_api.py::test_async_client_
batch_with_semaphore PASSED [ 88%] tests/test_api.py::test_async_client_retry_failure PASSED [10
0%] ============================= 9 passed in 0.88s ==============================
```

Рисунок 3 — Фактичний результат успішного прогону тестового сьюту pytest для ЛР8

13. Відповіді на контрольні запитання

1. Що таке клієнт-серверна архітектура та HTTP? — Клієнт-серверна архітектура — модель взаємодії, де клієнт надсилає запит, а сервер обробляє його і повертає відповідь. HTTP — протокол передавання гіпертексту прикладного рівня, який стандартизує методи, заголовки та формат тіла повідомлення.

2. У чому полягають відмінності методів PUT та PATCH? — Метод PUT призначений для повної заміни всього ресурсу (ідемпотентний). Метод PATCH виконує часткове оновлення лише окремих переданих полів ресурсу.

3. Призначення кодів стану 200, 201, 204, 404, 422: — 200 (OK): успішне читання/обробка; 201 (Created): ресурс створено; 204 (No Content): дія успішна, відповідь без тіла; 404 (Not Found): ресурс відсутній; 422 (Unprocessable Entity): помилка валідації Pydantic.

4. Що таке Event Loop та як працює async/await? — Event Loop — нескінченний цикл, який керує диспетчеризацією асинхронних задач у єдиному потоці. Ключове слово `await` сигналізує про призупинення виконання поточної корутини під час очікування I/O, передаючи контроль іншим операціям.

5. Різниця між asyncio.Task та asyncio.gather(): — `create_task()` негайно реєструє корутину в черзі виконання Event Loop як незалежну задачу; `gather()` очікує завершення переданого пулу задач і повертає агрегований кортеж результатів.

6. Навіщо обмежувати паралелізм через Semaphore? — Семафор виступає лічильником доступних слотів, обмежуючи число одночасних мережевих підключень. Це захищає клієнт і сервер від вичерпання дескрипторів сокетів та перевантаження пулу з'єднань БД.

7. Чим httpx.AsyncClient відрізняється від синхронного requests? — `AsyncClient` не блокує системний потік під час виконання мережевого I/O, підтримує connection pooling та взаємодіє з екосистемою `asyncio`.

14. Висновок

У ході виконання лабораторної роботи №8 було побудовано повнофункціональний асинхронний REST API для каталогу автомобілів (Варіант №5) з інтеграцією до шарів сервісів та бази даних ЛР7. Використання FastAPI та Pydantic v2 дозволило досягти автоматичної суворої валідації вхідних даних (року випуску, ціни, VIN) та автоматичної генерації специфікації OpenAPI (Swagger). Застосування інструментів `asyncio`, `asyncio.Semaphore`, `httpx.AsyncClient` та retry-механізмів забезпечило стабільну й контрольовану паралельну обробку потоків запитів до API. Надійність розробленої системи повністю підтверджено автоматизованими тестами `pytest` (9 passed).